Fall 2025

CS-UY 2413 (Sections A and B): Design and Analysis of Algorithms

Prof. Lisa Hellerstein
lisa.hellerstein@nyu.edu

Lecture 3

- Recommended Reading:
  - Chap. 3. A quick review will be done in class.
  - Next topic: Chap. 4

- Syllabus reminder
- Office Hours
- Homework rules reminder

- Last lecture, we were talking about big-Oh, big-Omega, little-oh, little-Omega, theta

- [Section A, skip to Polynomials of degree k slide]

Formal Definition of Big-Oh and Big-Omega

We assume that $f(n) > 0$ and $g(n) > 0$ for all $n > 0$

- $O(g(n)) = \{f(n) : \exists c > 0, \exists n_0 > 0 \text{ such that } \forall n \geq n_0, \ \ 0 \leq f(n) \leq cg(n)\}$
- $\Omega(g(n)) = \{f(n) : \exists c > 0, \exists n_0 > 0 \text{ such that } \forall n \geq n_0, \ \ 0 \leq cg(n) \leq f(n)\}$

Formal Definition of little-oh

We assume $f(n) > 0$ and $g(n) > 0$ for all $n > 0$

Recall definition of big-Oh: [intuitively, $f(n)$ is less than or equal to $g(n)$ (ignoring constants etc.)]

- $O(g(n)) = \{f(n) : \exists c > 0, \exists n_0 > 0$ such that $\forall n \geq n_0, 0 \leq f(n) \leq cg(n)\}$

Here is definition of little-Oh:

- $o(g(n)) = \{f(n) : \forall c > 0, \exists n_0 > 0$ such that $\forall n \geq n_0, \ \ 0 \leq f(n) < cg(n)\}$ [intuitively, $f(n)$ is much less than $g(n)$]

- Difference between definitions?



Formal Definition of little-oh

We assume $f(n) > 0$ and $g(n) > 0$ for all $n > 0$

Recall definition of big-Oh: [intuitively, $f(n)$ is less than or equal to $g(n)$ (ignoring constants etc.)]

- $O(g(n)) = \{f(n) : \exists c > 0, \exists n0 > 0$ such that $\forall n \geq n0, 0 \leq f(n) \leq cg(n)\}$

Here is definition of little-Oh:

- $o(g(n)) = \{f(n) : \forall c > 0, \exists n_0 > 0$ such that $\forall n \geq n_0, \ \ 0 \leq f(n) < cg(n)\}$ [intuitively, $f(n)$ is much less than $g(n)$]

- Difference between definitions?

main difference is $\forall c$ versus $\exists c$

- Under assumption that $f(n) > 0$ and g(n) > 0 when $n > 0$
  - $f(n) = o(g(n))$ **if**

$$lim_{n\to\infty}\frac{f(n)}{g(n)} = 0$$ [Limit test, limit needs to exist]

- e.g., $f(n) = 3n^2 + 4n + 189$ and $g(n) = n^3$

$$\lim_{n\to\infty} \frac{3n^2+4n+189}{n^3} = 0$$

$3n^2+4n+189 = o(n^3)$

- Could define ω analogously to how we defined o
- Instead, just quote following fact:
  - Under assumption that $f(n) > 0$ and g(n) > 0 when $n > 0$
    - $f(n) = o(g(n))$ iff $g(n) = \omega(f(n))$
- Similar fact:
  - Under assumption that $f(n) > 0$ and g(n) > 0 when $n > 0$
    - $f(n) = O(g(n))$ iff $g(n) = \Omega(f(n))$

Polynomials of degree k

- Example: $f(n) = 3n^{25} + 2n^9 + 4$
- then $f(n) = o(n^b)$ for all $b > 25$

  $= \omega(n^b)$ for all $b < 25$

  $= \Theta(n^b)$ for $b = 25$

Polynomials of degree k

- Claim: Suppose $f(n)$ is a polynomial of degree k
  $f(n) = a_k n^k + a_{k-1} n^{k-1} + \ldots + a_1 n^1 + a_0$ where $a_k > 0$
  and $f(n) > 0$ for all $n > 0$
  - then $f(n) = o(n^b)$ for all $b > k$

    $= \omega(n^b)$ for all $b < k$

    $= \Theta(n^b)$ for $b = k$

Properties of the relation

- Big-Oh relation on functions
  - Reflexive
    - $f(n) = O(f(n))$
  - Transitive
    - if $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$
  - But is it symmetric?

If $f(n) = O(g(n))$ does that imply $g(n) = O(f(n))$?

$n^2 = O(n^5)$ but $n^5 \neq O(n^2)$

NO.

Properties

- $f_1(n) = O(g(n))$ and $f_2(n) = O(g(n))$
  $\Rightarrow f_1(n) + f_2(n) = O(g(n))$

e.g. $f_1(n) = n^2 + 3 = O(n^2)$
$f_2(n) = 5n = O(n^2)$
$f_1(n) + f_2(n) = n^2 + 5n + 3 = O(n^2)$

- $f_1(n) = O(g(n)), f_2(n) = O(g(n)), \ldots, f_{254}(n) = O(g(n))$
  $\Rightarrow$

$f_1(n) + f_2(n) + \cdots + f_{254}(n) = O(g(n))$

Properties

- $f_1(n) = O(g(n))$ and $f_2(n) = O(g(n))$
  $\Rightarrow f_1(n) + f_2(n) = O(g(n))$
- More generally, for any constant $k > 0$,
  $f_1(n) = O(g(n)),\ f_2(n) = O(g(n)), \ldots,\ f_k(n) = O(g(n))$
  $\Rightarrow$ $f_1(n) + f_2(n) + \cdots + f_k(n) = O(g(n))$
- But, doesn't hold in general if k is not a constant!

  e.g. $\underbrace{n^2 + n^2 + n^2 + \cdots + n^2}_{n \text{ copies}} = n^3$

$f_1(n) = n^2$
$f_2(n) = n^2$
$\vdots$
$f_n(n) = n^2$
n copies
$k = n$

TERMINOLOGY BREAK

“Is stronger than”

- Which is a stronger statement?
  1. This movie is extremely long.
  2. This movie is long.

“Is stronger than”

- Which is a stronger statement?

  1. This movie is extremely long.
  2. This movie is long.

Say that Statement 1 *is stronger than* Statement 2 if
Statement 1 implies Statement 2

Returning to asymptotic notation…

Other properties

- Little-o "is stronger than" (implies) Big-Oh

$$f(n) = o(g(n)) \Rightarrow f(n) = O(g(n))$$

- Little-omega "is stronger than" (implies) Big-Omega

$$f(n) = \omega(g(n)) \Rightarrow f(n) = \Omega(g(n))$$

Shortcut

- Want to determine when $f(n)$, $g(n)$ positive valued whether
  $f(n) = \theta(g(n))$, $f(n) = o(g(n))$, or $f(n) = \omega(g(n))$
- Look at $\lim_{n\to\infty} \frac{f(n)}{g(n)}$
  - if limit = 0 then $f(n) = o(g(n))$
  - if limit = positive number then $f(n) = \Theta(g(n))$
  - if limit = ∞ then $f(n) = \omega(g(n))$

e.g. $f(n) = 3n^2 + 4n + 2$, $g(n) = n^2 + 1$

$\lim_{n\to\infty} \frac{3n^2+4n+2}{n^2+1} = 3$

Use of asymptotic notation

- Use of asymptotic notation when analyzing algorithms
  - $f(n) = O(n^2)$ means $f(n)$ is a member of the class $O(n^2)$
  - Doesn't behave like standard =

$6n^2 \in O(n^3)$

$7n^3 + 4n + 1 \in O(n^3)$

$6n^2 \neq 7n^3 + 4n + 1$

?

??

$x = y$

$z = y$

$\Rightarrow x = z$

- We may write, e.g.,

$$O(n^2) + 43n + 2$$

- What do we mean? $O(n^2)$ is a class of functions!

Means $g(n) + 43n + 2$

for some $g(n) = O(n^2)$

e.g. $g(n) = 24n^2 + 3n + 7$

Properties of some standard functions

Logarithms

Happy story

- You start off with $1
- Every day, you double your money.
  - Repeated doubling
- After *n* days, how much money do you have?

After 1 day: $2
2 days: $4
3 days: $8
4 days: $16

n days: 2^n

Another Happy Story

- You start off with $1
- Every day, you triple your money.
  - Repeated tripling
- After n days, how much money do you have?

$\$3^n$

Sad story

- You start off with n dollars
- Every day, you have to pay a bill that leaves you with only half your remaining money
  - (assume always have even number of dollars left)
  - Repeated halving
- How many days until you are down to only \$1? $\log_2 n$

Day x: $\frac{n}{2^x}$

Set equal to 1, solve for x

$$\frac{n}{2^x} = 1$$

$$\Rightarrow \quad n = 2^x$$

$$\Rightarrow \quad \log_2 n = \log_2 2^x$$

$$\log_2 n = x$$

Sad story

- You start off with n dollars
- Every day, you have to pay a bill that leaves you with only half your remaining money
  - (assume always have even number of dollars left)
  - Repeated halving
- How many days until you are down to only $1?

Relation to binary search?

Another sad story

- You start off with n dollars
- Every day, you have to pay a bill that leaves you with only a third of your remaining money
  - (assume always have ~~even~~ wide number of dollars left)
  - Repeated "thirding"
- How many days until you are down to only $1?

$\log_3 n$

Properties of logs and exponents

- For $a, b, c > 0$ and all n, assuming none of the logarithm bases are equal to 1
  - $b^{\log_b a} = a$ $\quad 3^{\log_3 7} = 7$
    - Because $\quad x = \log_b a \iff b^x = a$
  - $\log_b a = \dfrac{\log_c a}{\log_c b}$ change of base formula
  - $\log_b ac = \log_b a + \log_b c$ log of a product
  - $\log_b a^n = n \log_b a$ log of an exponential

More re logs and exponents

- $(a^b)^c = a^{bc}$
- $a^{\log_b c} = c^{\log_b a}$
  - Recall that $b^{\log_b a} = a$
  - So $a^{\log_b c} = (b^{\log_b a})^{\log_b c} = b^{(\log_b a)(\log_b c)} = b^{(\log_b c)(\log_b a)} = (b^{\log_b c})^{\log_b a} = c^{\log_b a}$
- Precedence rules
  - for exponentiation: a^{b^c} means $a^{(b^c)}$
  - for logs: $\log_b a + c$ means $(\log_b a) + c$

Recurrence Relations

- Consider a function $T(n)$ defined on the positive integers, with the following recursive definition:
  - ~~$T(1) = 1$~~ $T(n) = 1$ for $n = 1$ This is an example of a Recurrence Relation
  - $T(n) = T(n-1) + 3$ for all $n > 1$

- Consider a function $T(n)$ defined on the positive integers, with the following recursive definition:
  - $T(1) = 1$
  - $T(n) = T(n-1) + 3$ for all $n > 1$

What is $T(2)$? $T(1)+3 = 1+3 = 4$

What is $T(3)$? $T(2)+3 = 4+3 = 7$

- Consider a function $T(n)$ defined on the positive integers, with the following recursive definition:
  - $T(1) = 1$
  - $T(n) = T(n-1) + 3$ for all $n > 1$

More generally, what is $T(n)$?

Solve the recurrence relation. Give an arithmetic expression for $T(n)$.

- We will be using recurrence relations to express the running time of recursive algorithms

Divide and Conquer

Divide and Conquer Approach

- To solve problem, break it into smaller subproblems of same type
- Recursively solve the subproblems
- Combine solutions to subproblems to get solution to original problem

Mergesort

- Mergesort high-level idea (review from CS1134):
  - Divide array in half
  - Recursively sort both sides
  - Merge the two sorted sides

Mergesort

- Mergesort high-level idea (review from CS1134):
  - Divide array in half
  - Recursively sort both sides
  - Merge the two sorted sides

Base case?

Mergesort

- Mergesort high-level idea (review from CS1134):
  - Divide array in half
  - Recursively sort both sides
  - Merge the two sorted sides

Base case? array of length 0 or 1, do nothing

CLRS pseudocode for Mergesort

Sorts $A[p:r]$

```
MERGE-SORT(A, p, r)
1  if p ≥ r                          // zero or one element?
2      return
3  q = ⌊(p + r)/2⌋                   // midpoint of A[p : r]
4  MERGE-SORT(A, p, q)               // recursively sort A[p : q]
5  MERGE-SORT(A, q + 1, r)           // recursively sort A[q + 1 : r]
6  // Merge A[p : q] and A[q + 1 : r] into A[p : r].
7  MERGE(A, p, q, r)
```

↑ assumes $A[p:q]$ is sorted
$A[q+1:r]$ is sorted

Merges those 2 subarrays & puts result in $A[p:r]$

```
MERGE-SORT(A, p, r)
1  if p ≥ r                          // zero or one element?
2      return
3  q = ⌊(p + r)/2⌋                   // midpoint of A[p : r]
4  MERGE-SORT(A, p, q)               // recursively sort A[p : q]
5  MERGE-SORT(A, q + 1, r)           // recursively sort A[q + 1 : r]
6  // Merge A[p : q] and A[q + 1 : r] into A[p : r].
7  MERGE(A, p, q, r)
```

To sort the whole array A, we call `Merge-Sort(A, 1, n)`

Assume A is an array of length n.

Correctness of Mergesort

- Could do formally by induction on the size of the (sub)array
- But essentially, for divide and conquer algorithms, it's usually enough to show that:
  - Base of the recursion is correct
  - The work is divided correctly
  - Assuming the recursive calls are correct, their results are combined correctly
    - For merge sort, this means assuming the 2 recursive calls sort correctly, need to show that the merge procedure correctly merges them

Time for Mergesort

```
MERGE-SORT(A, p, r)
1  if p ≥ r                          // zero or one element?
2      return
3  q = ⌊(p + r)/2⌋                   // midpoint of A[p : r]
4  MERGE-SORT(A, p, q)               // recursively sort A[p : q]
5  MERGE-SORT(A, q + 1, r)           // recursively sort A[q + 1 : r]
6  // Merge A[p : q] and A[q + 1 : r] into A[p : r].
7  MERGE(A, p, q, r)
```

- Let $T(n)$ be worst-case runtime of Mergesort on array of length n
- What is the worst-case runtime of Mergesort on an array of length n? Express as *recurrence relation*.

$$T(n) = 2T\left(\frac{n}{2}\right) + \text{time to merge} \quad \text{if } n > 1$$

Time for Mergesort

```
MERGE-SORT(A, p, r)
1  if p ≥ r                          // zero or one element?
2      return
3  q = ⌊(p + r)/2⌋                   // midpoint of A[p : r]
4  MERGE-SORT(A, p, q)               // recursively sort A[p : q]
5  MERGE-SORT(A, q + 1, r)           // recursively sort A[q + 1 : r]
6  // Merge A[p : q] and A[q + 1 : r] into A[p : r].
7  MERGE(A, p, q, r)
```

- Let $T(n)$ be worst-case runtime of Mergesort on array of length n
- What is the worst-case runtime of Mergesort on an array of length n? Express as *recurrence relation.*

$$T(n) = 2T(n/2) + \theta(n) \quad \text{if } n > 1$$
$$T(n) = \theta(1) \quad \text{if } n = 1$$

Assume n a power of 2
Otherwise $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \theta(n)$

Recurrence relation for runtime of Mergesort

- $T(n) = 2T(n/2) + c_2 n \qquad \text{if } n > 1$
- $T(n) = c_1 \qquad \text{if } n = 1$

- Defines function in terms of itself
- Want to solve recurrence to get ``closed-form expression'' for runtime

Section 2.3 of textbook shows solution to recurrence for Mergesort

Recurrence relation for runtime of Mergesort

- $T(n) = 1$ if $n = 1$
- $T(n) = 2T(n/2) + n$ if $n > 1$
  - For simplicity, set $c_1 = c_2 = 1$
  - What is $T(8)$?

Top Down

$T(n) = 2T(\frac{n}{2}) + n$ $\quad\quad$ $T(1) = 1$

- $T(8) = 2T(4) + 8$ $= 2*12 + 8 = 32$
- $T(4) = 2T(2) + 4$ $= 2*4 + 4 = 12$
- $T(2) = 2T(1) + 2$ $= 2*1 + 2 = 4$
- $T(1) = 1$

Top Down

- $T(8) = 2T(4) + 8 = 32$
- $T(4) = 2T(2) + 4 = 12$
- $T(2) = 2T(1) + 2 = 4$
- $T(1) = 1$

Bottom Up (to begin with)

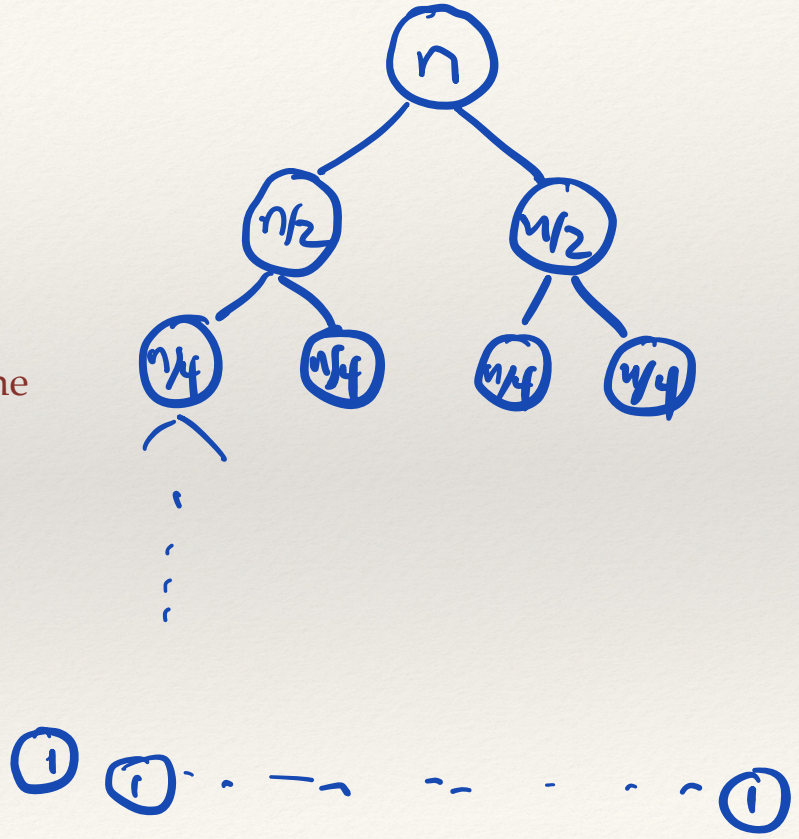
- $T(8) = 2T(4) + 8$ $= 2 \cdot 12 + 8 = 32$
- $T(4) = 2T(2) + 4$ $= 2 \cdot 4 + 4 = 12$
- $T(2) = 2T(2) + 2$ $= 2 \cdot 1 + 2 = 4$
- $T(1) = 1$



Recursion Tree method

- $T(n) = 1$ if $n = 1$
- $T(n) = 2T(n/2) + n$ if $n > 1$

Level #	Input size	#nodes	Time per node	Total Time
0	n	1	n	n
1	n/2	2	n/2	n
2	n/4	4	n/4	n
⋮				
i				



Recursion Tree method

- $T(n) = 1$ if $n = 1$
- $T(n) = 2T(n/2) + n$ if $n > 1$

Level #	Input size	#nodes	Time per node	Time in level
0	n	1	n	
1	n/2	2	n/2	
2	n/4	4	n/4	
⋮				
i	$n/2^i$	2^i	$n/2^i$	
⋮				
	2	$n/2$	2	
	1			

T(n)=

Recursion Tree method

- $T(n) = 1$ if $n = 1$
- $T(n) = 2T(n/2) + n$ if $n > 1$

Level #	Input size	#nodes	Time per node	Time in level
0	n	1	n	
1	n/2	2	n/2	
2	n/4	4	n/4	
⋮				
i	$n/2^i$	2^i	$n/2^i$	
⋮				
	2	$n/2$	2	
	1			

T(n)= $n \log_2 n + n$ Check it?

Recurrence relation for runtime of Mergesort

- $T(n) = c_1$ if $n = 1$
- $T(n) = 2T(n/2) + c_2 n$ if $n > 1$
  - ~~For simplicity, set $c_1 = c_2 = 1$~~
  - Re-do, with c_1 and c_2

Recursion Tree method

- $T(n) = c_1$ if $n = 1$
- $T(n) = 2T(n/2) + c_2 n$ if $n > 1$

Level #	Input size	#nodes	Time per node	Time in level
0	n	1	n	
1	n/2	2	n/2	
2	n/4	4	n/4	
⋮				
i	$n/2^i$	2^i	$n/2^i$	
⋮				
$\log_2 n - 1$	2	$n/2$	2	
$\log_2 n$	1	n	1	

